

JS

BEGINNER FRIENDLY GUIDE

JavaScript Promises Made Easy!

Pending...

Resolved!

Rejected

Learn how JavaScript handles tasks that take time,
explained in the simplest way possible!

Ages 10 - 14 | Beginners

Promises | .then() | .catch() | Async Basics | Practice Questions

What is a JavaScript Promise?

A Promise in JavaScript is like making a deal with someone. You say: "I promise to give you the answer... but I need a little time!"

Real-Life Example: Pizza Delivery

You order a pizza online. The app says: "Your order is confirmed!" Right now, you don't have the pizza yet - but you have a promise that it's coming! The delivery can succeed (pizza arrives) or fail (restaurant is closed). That's exactly how JavaScript Promises work!

In JavaScript, a Promise is used when something takes time - like loading a webpage, downloading a photo, or fetching data from the internet.

The 3 States of a Promise

Every Promise lives in one of 3 states. Think of it like tracking your delivery:

?

PENDING

Waiting... still working on it.
Like your pizza still being prepared!

OK

RESOLVED

Success! Everything went perfectly. Pizza arrived hot and fresh!

X

REJECTED

Something went wrong.
Restaurant closed, no pizza for you today!

Quick Memory Trick!

Pending = Still waiting | Resolved = Happy ending | Rejected = Oops, failed!

Your First Promise - JavaScript Code

Let's write a real JavaScript Promise! Don't worry - we'll explain every single line:

```
JavaScript

// We create a new Promise
// It takes a function with two tools:
//   resolve = call this if everything is OK
//   reject  = call this if something went wrong

const myPromise = new Promise( (resolve, reject) => {

    let pizzaDelivered = true;    // Let's pretend pizza arrived!

    if (pizzaDelivered) {
        resolve('Pizza is here! Enjoy your meal!'); // SUCCESS
    } else {
        reject('Sorry, we ran out of pizza!');      // FAILED
    }

}); // Promise is now created!
```

What just happened?

We created a Promise that checks if a pizza was delivered. If yes, it calls `resolve()` with a happy message. If no, it calls `reject()` with a sad message. Simple as that!

.then() and .catch() - Handling the Result

After creating a Promise, we need to tell JavaScript: "What should we do when it finishes?"

.then() - The Happy Path

Use `.then()` when the Promise succeeds (resolves). It's like saying: "When the pizza arrives, I'll eat it!"

.catch() - The Sad Path

Use `.catch()` when the Promise fails (rejects). It's like saying: "If no pizza, I'll order something else!"



```
// Using our promise from before...

myPromise
  .then( (message) => {
    console.log('SUCCESS:', message);
    // Prints: SUCCESS: Pizza is here! Enjoy your meal!
  })
  .catch( (error) => {
    console.log('FAILED:', error);
    // Prints: FAILED: Sorry, we ran out of pizza!
  });
```

Where Are Promises Used in Real Life?

Promises are used everywhere on the internet! Here are some real examples:

Loading a Website

When you open Instagram, your phone makes a Promise to load all the photos. If internet works = photos appear (resolve). No internet = error message (reject).

Getting Weather Data

A weather app promises to get today's temperature from the internet. Success = shows the weather. Failure = "Could not connect" message.

Uploading a Photo

When you upload a selfie, the app promises to save it to the server. Success = "Upload complete!" message. Failure = "Upload failed, try again!"

Setting a Timer

JavaScript can use promises to wait for time. Like "Promise to ring an alarm after 5 seconds." After 5 seconds, the promise resolves!

Summary - The Big Picture

One-Sentence Definition:

A JavaScript Promise is an object that represents a task that hasn't finished yet - it will either succeed (resolve) or fail (reject) in the future.

Term	Meaning	Real-Life Example
Promise	A task that will finish later	Ordering food online
Pending	Still waiting for the result	Food being cooked
Resolved	Task completed successfully!	Food delivered!

Rejected	Task failed with an error	Restaurant closed
.then()	Run this code on success	Eating the food
.catch()	Run this code on failure	Order something else

BONUS: Practice Questions

Test what you've learned! Try to answer these without looking back:

Q1. What are the 3 states of a JavaScript Promise?

Hint: Think about your pizza order...

Your Answer: _____

Q2. What method do you use when a Promise SUCCEEDS?

Hint: It starts with a dot...

Your Answer: _____

Q3. What method do you use when a Promise FAILS?

Hint: Think about catching something that falls!

Your Answer: _____

Fun Activity - Fill in the Blanks!

Complete the code below. Fill in the blanks with the correct words:

JavaScript

```
const cookingPromise = new _____ ( (resolve, reject) => {

    let foodReady = true;

    if (foodReady) {
        _____ ('Dinner is served!'); // Fill: resolve or reject?
    } else {
        _____ ('Cooking failed!'); // Fill: resolve or reject?
    }
});

cookingPromise
    ._____ ( msg => console.log(msg) ) // Fill: then or catch?
    ._____ ( err => console.log(err) ); // Fill: then or catch?
```

Answer Key (peek only after trying!)

Line 1: Promise | Line 6: resolve | Line 8: reject | Line 13: .then | Line 14: .catch

Great job learning JavaScript Promises!

You now understand one of the most important concepts in JavaScript.

Keep coding, keep learning, and keep making Promises!